Отчёт по практическим работам № 2.1–2.4

Разработка веб-приложения на PHP по паттерну MVC

1. Цель и ожидаемый результат

Цель работы: научиться оптимизировать разработку веб-приложений с помощью архитектурных паттернов, а именно разделить приложение на независимые части: Model (данные), View (внешний вид), Controller (логика обработки запроса) и Router (определение, какой контроллер вызвать).

Что должно делать приложение:

- показывать список статей из базы данных MySQL по адресу /;
- показывать отдельную статью по адресу /article/{id};
- возвращать корректную страницу 404 для несуществующих адресов и статей.

Структура работы

Работа	Тема	Что получилось
2.1	Маршрутизация	Единая точка входа index.php, класс Router, автозагрузка классов
2.2	Контроллер и представление	Классы Controller, View, шаблоны, общий layout.php
2.3	Модель и БД	База mvc_demo, класс Database (PDO), модель Article
2.4	Роутер и финал	Финальный dispatch, итоговый index.php, проверка всех сценариев

2. Теоретическая часть

MVC (Model–View–Controller) делит приложение на три модуля, которые можно менять независимо друг от друга.

Идею предложил Трюгве Реенскауг в конце 1970-х, она была реализована в Smalltalk-80.

Компонент	За что отвечает	Чего делать не должен
Model	Данные, бизнес-логика, SQL-запросы	Зависеть от View и Controller
View	Отображение данных (HTML)	Ходить в БД, менять модель, обрабатывать ввод
Controller	Принимает запрос, обращается к модели, выбирает View	Содержать SQL и HTML
Router	Сопоставляет URL и HTTP-метод с контроллером и действием	Содержать бизнес-логику

Как проходит запрос в итоговом приложении:

~ Браузер запрашивает, например, /article/2.

~ Apache через .htaccess отправляет запрос в index.php (если такого физического файла нет).

~ Router находит маршрут /article/{id} и вызывает ArticleController::show(2).

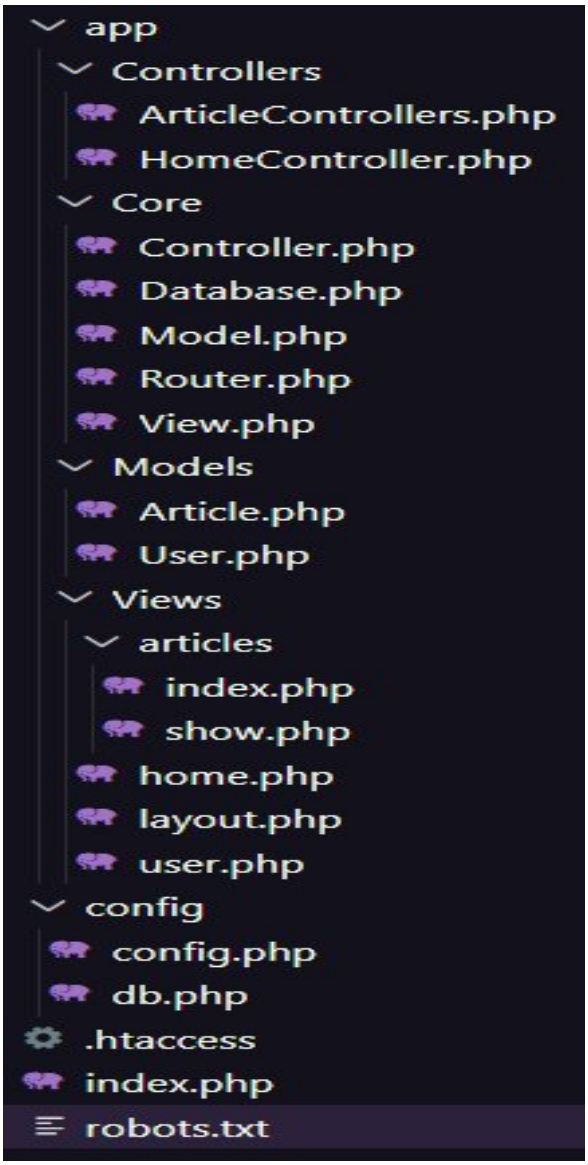
~ Контроллер просит у модели Article статью с id = 2.

~ Модель выполняет SQL-запрос через PDO и возвращает массив.

~ Контроллер передаёт массив во View, а тот подставляет его в шаблон articles/show.php.

~ Шаблон вставляется в layout.php, и готовый HTML уходит в браузер.

Структура проекта:



Пространство имён App\ соответствует папке app/. Например, класс App\Core\Router лежит в файле app/Core/Router.php.

Благодаря этому классы подключаются автоматически.

4. Практическая работа № 2.1. Роутер

4.1. Задание 1. Ручная маршрутизация

Что сделано. Создан index.php, который узнаёт, что именно запросил пользователь, и выводит нужную страницу через switch.

```
<?php
declare(strict_types=1);

// Что вообще пришло?
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$uri = rtrim($uri, '/') ?: '/';

echo "<pre>";
echo "REQUEST_URI: {$_SERVER['REQUEST_URI']}\n";
echo "PATH:        {$uri}\n";
echo "METHOD:      {$_SERVER['REQUEST_METHOD']}\n";
echo "</pre>";

// Простейшая маршрутизация вручную
switch ($uri) {
    case '/':
        echo "<h1>Главная страница — список статей</h1>";
        break;
    case '/about':
        echo "<h1>О проекте</h1>";
        break;
    default:
        if (preg_match('#^/article/(\d+)$#', $uri, $m)) {
            echo "<h1>Статья №{$m[1]}</h1>";
        } else {
            http_response_code(404);
            echo "<h1>404 — страница не найдена</h1>";
        }
}
```

Разбор кода:

- $_SERVER['REQUEST_URI'] содержит адрес вместе с параметрами (/article/5?x=1).
- parse_url(..., PHP_URL_PATH) оставляет только путь (/article/5).

- rtrim($uri, '/') ?: '/' убирает завершающий слэш, чтобы /about/ и /about считались одним адресом. Для корня результат пустой строки заменяется обратно на /.
- preg_match('#^/article/(\d+)$#', ...) проверяет, что после /article/ идут только цифры, и сохраняет число в $m[1].
- http_response_code(404) отправляет браузеру честный статус 404, а не просто текст.

Файл .htaccess:

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php [QSA,L]
```

Правило означает: если запрошенный путь не является реальным файлом (!-f) или папкой (!-d), то отдать запрос скрипту index.php.

Флаг QSA сохраняет параметры строки запроса, L прекращает обработку правил.

Для работы нужен включённый модуль mod_rewrite и AllowOverride All в настройках Apache.

Результаты проверки:

Адрес	**Результат**	**Код**
/	«Главная страница — список статей»	200
/about	«О проекте»	200
/article/5	«Статья №5»	200
/article/abc	«404 — страница не найдена» (abc не соответствует \d+)	404
/nope	«404 — страница не найдена»	404

Ответы на теоретические вопросы:

- Куда идут все запросы?

Все они попадают в один файл, index.php. Благодаря .htaccess Apache не ищет файлы about или article/5, а передаёт запрос единой точке входа. Этот приём называется Front Controller. Дальше скрипт сам решает, что показать.

~ Что будет, если маршрутов станет 50?

switch превратится в длинную нечитаемую простыню. Каждый маршрут придётся описывать вручную, цепочки preg_match будут дублироваться, а ошибки (например, перепутанный break) станет трудно найти. Файл index.php займётся всем сразу: и разбором адреса, и выводом страниц.

~ А если добавим POST?

switch смотрит только на адрес и не различает HTTP-методы. Но GET /article/5 (прочитать) и POST /article/5 (изменить) это разные действия. Пришлось бы вкладывать проверки метода в каждый case. Нужна структура, где маршрут определяется парой «метод + адрес».

4.2. Задание 2. Класс Router

Что сделано. Логика маршрутизации вынесена в отдельный класс app/Core/Router.php.

```
<?php
namespace App\Core;

class Router
{
    private array $routes = [];

    public function get(string $path, callable|array $handler):
void
    {
        $this->routes['GET'][$path] = $handler;
    }

    public function post(string $path, callable|array $handler):
void
    {
        $this->routes['POST'][$path] = $handler;
    }

    public function dispatch(string $method, string $uri): void
    {
        $uri = parse_url($uri, PHP_URL_PATH);
        $uri = rtrim($uri, '/') ?: '/';

        foreach ($this->routes[$method] ?? [] as $route =>
$handler) {
            $pattern = '#^' . preg_replace('#\{(\w+)\}#',
'(\d+)', $route) . '$#';

            if (preg_match($pattern, $uri, $matches)) {
                array_shift($matches);

                if (is_array($handler)) {
                    [$class, $action] = $handler;
                    (new $class())->$action(...$matches);
                } else {
                    $handler(...$matches);
                }
                return;
            }
        }
```

```
        http_response_code(404);
        echo '404 Not Found';
    }
}
```

Как работает класс:

- get() и post() регистрируют маршрут, то есть записывают обработчик в массив $routes.
- dispatch() выполняет маршрутизацию: берёт метод и адрес запроса, перебирает маршруты нужного метода и ищет совпадение.
- Запись {id} в маршруте превращается регулярным выражением в (\d+). Так /article/{id} становится шаблоном #^/article/(\d+)$#.
- array_shift($matches) убирает из результата preg_match нулевой элемент (всю найденную строку), и остаются только захваченные параметры.
- Обработчик может быть замыканием или массивом [Класс, 'метод']. Во втором случае роутер сам создаёт объект контроллера и вызывает действие. Оператор ...$matches раскрывает массив параметров в аргументы вызова.
- Если ничего не подошло, отправляется статус 404.

Переписанный index.php:

```
<?php
declare(strict_types=1);

spl_autoload_register(function (string $class) {
    $prefix = 'App\\';
    $baseDir = __DIR__ . '/app/';

    if (!str_starts_with($class, $prefix)) return;

    $file = $baseDir . str_replace('\\', '/', substr($class,
strlen($prefix))) . '.php';
    if (file_exists($file)) require $file;
});
```

```
use App\Core\Router;

$router = new Router();

// Пока — замыкания, контроллеров ещё нет
$router->get('/', fn() => print '<h1>Главная — список
статей</h1>');
$router->get('/about', fn() => print '<h1>О проекте</h1>');
$router->get('/article/{id}', fn($id) => print "<h1>Статья
№{$id}</h1>");

$router->dispatch($_SERVER['REQUEST_METHOD'],
$_SERVER['REQUEST_URI']);
```

Автозагрузчик (spl_autoload_register) превращает имя класса в путь к файлу: App\Core\Router → app/Core/Router.php. Благодаря ему не нужны ручные require для каждого класса.

Ответы на теоретические вопросы:

- Что изменилось?

Маршруты теперь описываются декларативно (одной строкой на маршрут), а не размазаны по switch и if.

Логика поиска совпадений спрятана в классе Router, а index.php стал коротким и читаемым.

Появилась поддержка разных HTTP-методов и автозагрузка классов.

- Что делает роутер?

Принимает HTTP-запрос (метод и URL), находит подходящий маршрут, извлекает параметры из адреса и передаёт управление нужному обработчику (контроллеру и его методу).

Если маршрута нет, возвращает 404.

Сам роутер ничего не знает о том, что именно делать со статьёй.

- Кто должен решать, что делать на /article/{id}?

Контроллер. Роутер только говорит «этот адрес обрабатывает такой-то метод такого-то класса», а само решение (найти статью, показать её или вернуть ошибку) принимает контроллер.

~ Что хранится в массиве $routes?

Двухуровневый ассоциативный массив вида $routes[HTTP-метод][шаблон адреса] = обработчик.

Например:

```
$routes = [
    'GET' => [
        '/'             => [ArticleController::class, 'index'],
        '/article/{id}' => [ArticleController::class, 'show'],
    ],
    'POST' => [],
];
```

Обработчиком может быть замыкание или пара [имя класса, имя метода].

5. Практическая работа № 2.2. Контроллер и представление

5.1. Задание 1. Первый контроллер

Что сделано. Создан базовый абстрактный класс app/Core/Controller.php (пока пустой) и контроллер app/Controllers/ArticleController.php, который наследует его. Данные пока «зашиты» в код, потому что базы данных ещё нет.

```
<?php
namespace App\Core;

abstract class Controller
{
    // Пока пустой — наполним на этапе View
}


<?php
namespace App\Controllers;

use App\Core\Controller;

class ArticleController extends Controller
{
    public function index(): void
    {
        // Захардкоженные данные — БД ещё нет!
        $articles = [
            ['id' => 1, 'title' => 'Первая статья'],
            ['id' => 2, 'title' => 'Вторая статья'],
            ['id' => 3, 'title' => 'Третья статья'],
        ];

        echo '<h1>Все статьи</h1>';
        foreach ($articles as $a) {
            echo "<p>{$a['title']} — <a
href='/article/{$a['id']}'>читать</a></p>";
        }
    }

    public function show(int $id): void
    {
        echo "<h1>Статья #{$id}</h1>";
        echo '<p>Здесь будет содержимое статьи.</p>';
        echo '<p><a href="/">← Назад</a></p>';
    }
```

Маршруты в index.php теперь указывают на контроллер:

```
$router->get('/', [ArticleController::class, 'index']);
$router->get('/article/{id}', [ArticleController::class, 'show']);
```

Ответы на теоретические вопросы:

- Что делает контроллер? Это связующее звено: принимает запрос от роутера, получает нужные данные (из модели), решает, какое представление показать, и передаёт данные туда. Хороший контроллер «тонкий»: в нём несколько десятков строк, без SQL и без HTML.
- Что мы сделали неправильно? Что не так с этим echo?
- Смешали логику и отображение. HTML-разметка написана прямо в контроллере, хотя по правилам MVC она должна быть в представлении.
- Данные захардкожены в контроллере, а должны приходить из модели.
- Нет экранирования. Строка {$a['title']} выводится как есть. Если в заголовке окажется <script>...</script>, браузер его выполнит, то есть возникнет уязвимость XSS. Нужен htmlspecialchars().
- Нет общего каркаса страницы. Нет <html>, <head>, <body>, кодировки и стилей, а дизайнеру негде менять оформление, не трогая PHP-код.
- Каждый метод сам печатает вывод, поэтому код нельзя повторно использовать и сложно тестировать.

5.2. Задание 2. Представление (View)

Идея: контроллер готовит данные, View их рисует.

app/Core/View.php

```
<?php
namespace App\Core;

class View
{
    public static function render(string $template, array $data = []): void
    {
        extract($data); // $articles / $article станут переменными

        $viewFile = __DIR__ . '/../Views/' . $template . '.php';
        if (!file_exists($viewFile)) {
            http_response_code(500);
            exit("View not found: {$template}");
        }

        ob_start();
        require $viewFile;
        $content = ob_get_clean();  // содержимое шаблона

        require __DIR__ . '/../Views/layout.php'; // обёртка
    }
}
```

Как работает render():

1) extract($data) превращает ключи массива в переменные: ['articles' => [...]] даёт переменную $articles в шаблоне.
2) Проверяется, что файл шаблона существует. Если нет, возвращается ошибка 500.
3) ob_start() включает буферизацию вывода: всё, что печатает шаблон, попадает не в браузер, а в память.
4) require $viewFile подключает шаблон, и его вывод оказывается в буфере.

5) ob_get_clean() забирает накопленный HTML в переменную $content и очищает буфер.
6) Подключается layout.php, который вставляет $content внутрь общей обёртки.
7) app/Core/Controller.php (наполняем): добавлен вспомогательный метод view(), чтобы наследники не вызывали View::render напрямую.

```
<?php
namespace App\Core;

abstract class Controller
{
    protected function view(string $template, array $data = []):
void
    {
        View::render($template, $data);
    }
}
```

app/Views/layout.php (общий шаблон):

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>MVC Demo Blog</title>
    <style>
        body { font-family: sans-serif; max-width: 800px; margin:
40px auto; padding: 0 20px; }
        .article { border-bottom: 1px solid #eee; padding: 15px 0; }
        a { color: #0066cc; text-decoration: none; }
        a:hover { text-decoration: underline; }
    </style>
</head>
<body>
    <header>
        <h1><a href="/"> MVC Blog</a></h1>
        <hr>
    </header>
    <main><?= $content ?></main>
</body>
</html>
```

app/Views/articles/index.php (список):

```
<h2>Все статьи</h2>
<?php foreach ($articles as $article): ?>
    <div class="article">
        <h3><?= htmlspecialchars($article['title']) ?></h3>
        <a href="/article/<?= $article['id'] ?>">Читать →</a>
    </div>
<?php endforeach; ?>
```

app/Views/articles/show.php (одна статья):

```
<a href="/">← Назад</a>
<h2><?= htmlspecialchars($article['title']) ?></h2>
<p><?= nl2br(htmlspecialchars($article['content'])) ?></p>
```

В шаблонах используется htmlspecialchars(), который превращает <, >, & и кавычки в безопасные сущности и защищает от XSS. Функция nl2br() превращает переводы строк в теги
. Запись <?= ... ?> это короткая форма <?php echo ... ?>. Переменная $content в layout.php не экранируется намеренно: это уже готовый и проверенный HTML шаблона.

app/Controllers/ArticleController.php (после рефакторинга, данные пока заглушки):

```
<?php
namespace App\Controllers;

use App\Core\Controller;

class ArticleController extends Controller
{
    public function index(): void
    {
        $articles = [
            ['id' => 1, 'title' => 'Первая статья', 'content' => '...'],
            ['id' => 2, 'title' => 'Вторая статья', 'content' => '...'],
            ['id' => 3, 'title' => 'Третья статья', 'content' => '...'],
        ];
        $this->view('articles/index', ['articles' => $articles]);
    }

    public function show(int $id): void
    {
        $article = [
            'id'      => $id,
```

```
            'title'   => "Статья #{$id}",
            'content' => 'Здесь будет содержимое статьи. Пока это заглушка.',
        ];
        $this->view('articles/show', ['article' => $article]);
    }
}
```

Примечание: в методичке этот файл ошибочно подписан как app/Core/Controller.php. Правильный путь: app/Controllers/ArticleController.php (об этом говорит и его namespace App\Controllers).

Ответы на теоретические вопросы:

- Что именно делает контроллер после рефакторинга? Чем он занят, если убрали HTML? Он только подготавливает данные (пока из массива, дальше из модели) и вызывает $this->view(...), выбирая шаблон и передавая данные. Контроллер управляет потоком, но не рисует страницу.
- Что осталось в контроллере, а что ушло в шаблон? В контроллере остались получение данных, выбор шаблона и обработка ситуаций вроде «статья не найдена». В шаблон ушла вся HTML-разметка, цикл foreach для вывода и экранирование.
- Почему layout.php один, а шаблонов может быть много? layout.php содержит то, что одинаково на всех страницах: <html>, <head>, стили, шапку. А содержимое (список, одна статья, форма) у каждой страницы своё. Общую часть пишут один раз (принцип DRY), а меняющуюся подставляют в неё.
- Как layout.php связан с ob_start() и ob_get_clean()? Сначала шаблон страницы выполняется «молча»: ob_start() перехватывает его вывод, а ob_get_clean() кладёт готовый HTML в переменную $content. Затем подключается layout.php, который печатает свою обёртку и в месте <?=

$content ?> вставляет результат шаблона. Без буферизации шаблон напечатался бы раньше, чем <head>, и страница сломалась бы.

- Что было бы, если бы каждый шаблон содержал свой <html>, <head> и <body>? Код дублировался бы в каждом файле, стили и мета-теги пришлось бы копировать, и при расхождении страницы выглядели бы по-разному. Любая правка шапки или подключаемых стилей требовала бы изменить все шаблоны сразу, а вероятность ошибки росла бы с каждой новой страницей.

- Что будет, если завтра дизайнер захочет поменять меню? Достаточно отредактировать один файл, layout.php, и изменение сразу появится на всех страницах. Контроллеры, модели и остальные шаблоны трогать не нужно.

- А откуда взять статьи, если их станет 10 000? Хранить в массиве в коде невозможно, поэтому нужна база данных и модель, которая делает SQL-запросы. Выводить все 10 000 записей на одну страницу тоже неразумно, так что понадобится постраничный вывод (LIMIT/OFFSET), индексы в таблице и, возможно, кэширование. Это тема следующей работы.

6. Практическая работа № 2.3. Модель и база данных

6.1. Задание 1. База данных и модель

Что сделано. Создана база данных mvc_demo с таблицей articles, класс подключения Database и модель Article. Теперь SQL живёт только в модели.

SQL-скрипт:

```
CREATE DATABASE mvc_demo CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE mvc_demo;

CREATE TABLE articles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO articles (title, content) VALUES
('Первая статья', 'Это содержимое первой статьи про MVC.'),
('Вторая статья', 'Здесь мы разбираем контроллеры.'),
('Третья статья', 'А тут — про представления.');
```

Структура таблицы articles:

Поле	**Тип**	**Назначение**
id	INT, AUTO_INCREMENT, PRIMARY KEY	Уникальный номер статьи, присваивается автоматически
title	VARCHAR(255), NOT NULL	Заголовок
content	TEXT, NOT NULL	Текст статьи
created_at	TIMESTAMP, по умолчанию текущее время	Дата создания

Кодировка utf8mb4 нужна, чтобы корректно хранить русский текст (и эмодзи).

config/db.php (настройки подключения вынесены из кода):

```
<?php
return [
    'host'     => 'localhost',
    'dbname'   => 'mvc_demo',
```

```
    'user'     => 'root',
    'password' => '',
    'charset'  => 'utf8mb4',
];
```

app/Core/Database.php (одно подключение на всё приложение):

```
<?php
namespace App\Core;

use PDO;

class Database
{
    private static ?PDO $pdo = null;

    public static function pdo(): PDO
    {
        if (self::$pdo === null) {
            $config = require __DIR__ . '/../../config/db.php';
            $dsn =
"mysql:host={$config['host']};dbname={$config['dbname']};charset={$con
fig['charset']}";

            self::$pdo = new PDO($dsn, $config['user'],
$config['password'], [
                PDO::ATTR_ERRMODE            =>
PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            ]);
        }
        return self::$pdo;
    }
}
```

Это шаблон Singleton (одиночка): статическое свойство $pdo хранит единственное подключение. При первом вызове оно создаётся, при последующих просто возвращается. Так приложение не открывает новое соединение на каждый запрос к базе. Две опции PDO важны для работы:

ERRMODE_EXCEPTION: ошибки БД выбрасывают исключения, а не молча игнорируются;

FETCH_ASSOC: строки возвращаются как ассоциативные массивы ($row['title']), без дублирования числовых индексов.

app/Models/Article.php:

```
<?php
namespace App\Models;

use App\Core\Database;

class Article
{
    public function all(): array
    {
        return Database::pdo()
            ->query("SELECT * FROM articles ORDER BY created_at DESC")
            ->fetchAll();
    }

    public function find(int $id): ?array
    {
        $stmt = Database::pdo()->prepare("SELECT * FROM articles WHERE
id = ?");
        $stmt->execute([$id]);

        return $stmt->fetch() ?: null;
    }
}
```

all() возвращает все статьи, от новых к старым.

find($id) возвращает одну статью или null, если такой нет (fetch() при отсутствии строки даёт false, а ?: null превращает его в null, как и обещает тип ?array).

Используется подготовленный запрос (prepare + execute с ?). Значение $id передаётся отдельно от текста SQL, поэтому SQL-инъекция невозможна. Склеивать запрос вручную ("... WHERE id = $id") нельзя.

Финальный ArticleController:

```
<?php
namespace App\Controllers;

use App\Core\Controller;
use App\Models\Article;

class ArticleController extends Controller
{
    private Article $model;

    public function __construct()
    {
        $this->model = new Article();
    }

    public function index(): void
    {
        $this->view('articles/index', [
            'articles' => $this->model->all(),
        ]);
    }

    public function show(int $id): void
    {
        $article = $this->model->find($id);

        if (!$article) {
            http_response_code(404);
            exit('Статья не найдена');
        }

        $this->view('articles/show', ['article' => $article]);
    }
}
```

Контроллер в конструкторе создаёт модель. Метод index() берёт все статьи и отдаёт их шаблону, метод show() ищет статью и при неудаче возвращает 404.

Выводы по заданию (ответы на теоретические вопросы):

Форма вызова view() не изменилась. Контроллер по-прежнему отдаёт шаблону массив. Поменялся только источник данных: был захардкоженный массив, стала база данных. Ни шаблоны, ни View, ни роутер менять не пришлось.

Модель — единственное место, где живёт SQL. Контроллер не знает, как устроена база, а шаблон не знает, откуда пришли данные.

Замена хранилища затрагивает только модель. Если завтра статьи будут лежать в JSON-файле или приходить из внешнего API, достаточно переписать методы all() и find(). Контроллер и представления останутся как есть. Это и есть главное преимущество MVC: независимость компонентов.

6.2. Задание 2

В методичке второе задание повторяет первое (тот же SQL-скрипт, конфигурация, Database, Article и контроллер). Поэтому оно выполнено в рамках того же проекта, дублирующих файлов не создавалось. Результат один и тот же: данные берутся из базы, а все SQL-запросы собраны в модели.

7. Практическая работа № 2.4. Роутер и финальная сборка

Что сделано. Роутер доработан до финальной версии, а index.php собран в итоговый вид. Метод dispatch() сам создаёт контроллер (new $class()) и вызывает нужный метод. Это возможно, потому что у контроллера конструктор без обязательных аргументов.

Финальный dispatch() (app/Core/Router.php): код совпадает с версией из раздела 4.2 (разбор адреса, поиск маршрута для HTTP-метода, превращение {id} в (\d+), вызов обработчика, 404 при отсутствии совпадений).

Финальный index.php:

```
<?php
declare(strict_types=1);

spl_autoload_register(function (string $class) {
    $prefix = 'App\\';
    $baseDir = __DIR__ . '/app/';

    if (!str_starts_with($class, $prefix)) return;

    $file = $baseDir . str_replace('\\', '/', substr($class,
strlen($prefix))) . '.php';
    if (file_exists($file)) require $file;
});

use App\Core\Router;
use App\Controllers\ArticleController;

$router = new Router();

$router->get('/', [ArticleController::class, 'index']);
$router->get('/article/{id}', [ArticleController::class, 'show']);

$router->dispatch($_SERVER['REQUEST_METHOD'],
$_SERVER['REQUEST_URI']);
```

Весь index.php теперь состоит из трёх частей: автозагрузка классов, регистрация маршрутов, запуск роутера. Бизнес-логики в нём нет.

Пошаговый разбор запроса GET /article/2:

Шаг	Кто	Что происходит
1	Apache + .htaccess	Файла /article/2 нет, запрос уходит в index.php
2	index.php	Подключает автозагрузчик, регистрирует маршруты, вызывает dispatch('GET', '/article/2')
3	Router	Находит /article/{id}, получает ['2']
4	Router	Создаёт ArticleController и вызывает show('2')
5	ArticleController	Просит у Article статью с id = 2
6	Article	Выполняет SELECT ... WHERE id = ? через PDO, возвращает массив
7	ArticleController	Вызывает $this->view('articles/show', ['article' => ...])
8	View	Подставляет данные в шаблон, шаблон вкладывается в layout.php
9	Браузер	Получает готовую HTML-страницу

Почему show(int $id) работает, если роутер передаёт строку '2'? В index.php включён strict_types=1, но он действует только на вызовы, написанные в самом файле. Вызов ->$action(...$matches) находится в Router.php, где строгой типизации нет, поэтому PHP спокойно приводит строку '2' к int.

8. Итоговое тестирование

Проверены сценарии из задания 2.4 и граничные случаи.

№	Запрос	Ожидаемый результат	Код ответа	Итог
1	GET /	Страница «Все статьи»: список из 3 статей из БД со ссылками «Читать →»	200	Пройден
2	GET /article/1	Статья «Первая статья» с текстом из БД и ссылкой «← Назад»	200	Пройден
3	GET /article/2	Статья «Вторая статья»	200	Пройден
4	GET /article/999	Текст «Статья не найдена»	404	Пройден
5	GET /article/abc	404 Not Found (маршрут требует цифры)	404	Пройден
6	GET /nope	404 Not Found	404	Пройден
7	GET /article/1/ (слэш в конце)	Статья 1, слэш отбрасывается через rtrim	200	Пройден
8	POST /	404 Not Found (POST-маршрутов не зарегистрировано)	404	Пройден

[Вставьте сюда скриншоты страниц /, /article/1 и /article/999.]

Что проверялось в БД: при GET / выполняется один запрос SELECT * FROM articles ORDER BY created_at DESC, при GET /article/N один подготовленный запрос SELECT * FROM articles WHERE id = ?.

9. Общий вывод

В ходе работ № 2.1–2.4 реализовано веб-приложение на PHP с архитектурой MVC. Освоены:

Front Controller и перенаправление запросов через .htaccess;

Маршрутизация: класс Router с регистрацией маршрутов по HTTP-методам, параметрами в адресе ({id}) и обработкой 404;

Автозагрузка классов по стандарту PSR-4-подобной схемы (spl_autoload_register);

Контроллеры, которые только координируют работу и не содержат ни SQL, ни HTML;

Представления с общим шаблоном layout.php, буферизацией вывода (ob_start, ob_get_clean) и экранированием данных;

Модель с подготовленными запросами PDO и подключением к БД в виде Singleton.

Главный практический результат: каждый слой можно менять независимо. Дизайнер правит layout.php, не трогая PHP-логику, а замена MySQL на другое хранилище затрагивает только модель. Цель работы, оптимизация разработки за счёт архитектурного паттерна, достигнута.

10. Список источников

Что такое MVC: https://blog.skillfactory.ru/glossary/mvc/

Официальная русская документация PHP: https://www.php.net/manual/ru/

Учебник по PHP (Metanit): https://metanit.com/php/tutorial

Ещё один способ реализации MVC (Хабр): https://habr.com/ru/articles/150267/